

What is Docker?

Containers explained — from "works on my machine" to production

Press Space or → to advance

"Works on my machine"

The classic developer problem

Developer A sends code to Developer B

- └ B runs it → ❌ ImportError: No module named 'flask'
- └ B installs flask → ❌ Wrong Python version
- └ B fixes Python → ❌ Different OS behaviour
- └ Days later... "it works on my machine 🙄"

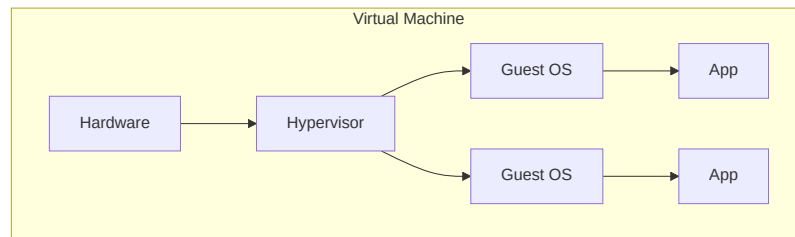
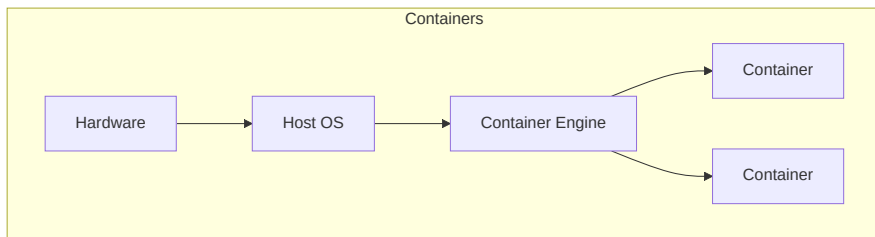
What we really want to ship

Ship the environment, not just the code.

Docker solves this by packaging your app + its runtime + its dependencies into a single portable unit called a container.

VMs vs Containers

Why containers are lighter



VMs

Containers

Boot time

Minutes

Milliseconds

Size

GBs

MBs

OS

Full guest OS

Shared host kernel

Isolation

Hardware-level

Namespace / cgroups

Core Concepts

The Docker vocabulary

-  **Image** — a read-only template. Like a recipe or a class definition. Built from a `Dockerfile`. Stored in a registry.
-  **Container** — a running instance of an image. Like an object instantiated from a class. Ephemeral by default.
-  **Dockerfile** — a text file of instructions that tells Docker how to build an image, step by step.
-  **Registry** — a storage and distribution server for images. `Docker Hub` is the public default; you can run your own.
-  **Layers** — each instruction in a Dockerfile adds a read-only layer. Layers are cached, so rebuilds are fast.

Anatomy of a Dockerfile

Every line is an instruction that adds a layer

```
# Start from an official base image
FROM python:3.12-slim

# Set the working directory inside the container
WORKDIR /app

# Copy dependency list and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application source
COPY . .

# Expose the port the app listens on
EXPOSE 5000

# Default command to run when the container starts
CMD ["python", "app.py"]
```

Each `RUN` / `COPY` / `ADD` creates a **cached layer** — unchanged layers are reused on rebuild.

The Build → Run → Ship Loop

Three commands you'll use every day

```
# 1. Build an image from the Dockerfile in the current directory
```

```
docker build -t my-app:latest .
```

```
# 2. Run a container from that image
```

```
docker run -p 5000:5000 my-app:latest
```

```
# 3. Push the image to a registry so others can pull it
```

```
docker push myusername/my-app:latest
```

```
# Pull and run an image someone else published
```

```
docker pull nginx:alpine
```

```
docker run -p 8080:80 nginx:alpine
```

```
# List running containers
```

```
docker ps
```

```
# Stop a container
```

```
docker stop <container-id>
```

```
# List local images
```

```
docker images
```

Volumes & Networks

Why data is ephemeral — and how to persist it

Problem: containers are ephemeral — when a container stops, all data written inside it is gone.

Solution: Volumes — mount a host path or a named Docker volume into the container.

```
# Named volume — Docker manages the path
docker run -v my-data:/var/lib/postgresql/data postgres

# Bind mount — maps a host directory directly
docker run -v $(pwd)/data:/app/data my-app
```

Networks — containers on the same Docker network reach each other by service name, not IP.

```
docker network create my-net
docker run --network my-net --name redis redis:7-alpine
docker run --network my-net my-app # app resolves "redis" as hostname
```

Docker Compose

Define multi-container apps in one file

```
# compose.yaml
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - redis
    environment:
      - REDIS_URL=redis://redis:6379

  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data

volumes:
  redis-data:
```

```
docker compose up --build    # build + start all services
docker compose down         # stop + remove containers
docker compose logs -f web   # tail logs for the web service
```

Multi-Stage Builds

Small, secure final images

```
# Stage 1: Builder — has the full toolchain
FROM golang:1.22 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app .

# Stage 2: Runtime — only what's needed to run
FROM gcr.io/distroless/static:nonroot
COPY --from=builder /app /app
EXPOSE 8080
ENTRYPOINT ["/app"]
```

- The **builder** stage has `gcc` , `go` `toolchain` , source code — hundreds of MB
- The **final** image has only the compiled binary — often < 10 MB
- Attack surface shrinks dramatically — no shell, no package manager
- CI builds get caching on the builder stage for free

Live Demo Projects

Three apps in this repo — each shows a different Docker pattern

App	Language	Docker Pattern	Backing Service
<code>flask-app/</code>	Python	Dockerfile + Compose	Redis (visit counter)
<code>go-app/</code>	Go	Multi-stage + Compose	Postgres
<code>rust-app/</code>	Rust	Multi-stage Dockerfile only	—

```
# Flask: multi-container via Compose
cd flask-app && docker compose up --build
# → http://localhost:5000
```

```
# Go: multi-stage image + Compose
cd go-app && docker compose up --build
# → http://localhost:8080
```

```
# Rust: build and run manually
cd rust-app
docker build -t rust-app .
```

Recap

What you learned today

-  Docker packages apps + dependencies into portable containers
-  Images are built from **Dockerfiles**; containers are running instances of images
-  Dockerfile layers are cached — order instructions from least → most frequently changed
-  Docker Compose orchestrates multi-container apps with a single YAML file
-  Multi-stage builds give you tiny, secure production images
-  Volumes persist data; **networks** let containers talk to each other by name

Further reading

- docs.docker.com — official documentation
- [Docker Getting Started tutorial](#)
- [Best practices for writing Dockerfiles](#)
- [Compose file reference](#)
- [dive](#) — explore image layers interactively